Chapter 5

Render Native Screens with SwiftUI

In the previous chapter, we added native tabs to swap between two web views and bring back our hidden web-based navigation links. But tabs are only a small portion of the screen, located all the way at the bottom. Sometimes we want to forgo the web entirely and render the entire UI with a more user-friendly, native implementation.

A major benefit of Hotwire Native is being able to do just this—to *drop down* into native Swift when needed. This unlocks access to the latest iOS APIs and provides developers with a way to render fully native screens.

In this chapter, you will learn how to decide which screens to upgrade to native vs. which are better rendered in the web view. Then, you'll build a native screen with SwiftUI in iOS.

There's a lot to cover when going native. So, to help keep the book flowing, I've split up the discussion of how to render native screens on iOS and Android between this chapter and the next one, Chapter 6, *Render Native Screens with Jetpack Compose*.

Before we dive into SwiftUI code, let's first review *when* it makes sense to go fully native. Each native screen adds additional complexity and maintenance, so you'll want to reserve them for the highlights of your app.